МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний аерокосмічний університет
«Харківський авіаційний інститут»

Факультет систем управління літальних апаратів
Кафедра систем управління літальних апаратів

# Лабораторна робота № 8

з дисципліни «Алгоритмізація та програмування»
на тему «Реалізація алгоритмів сортування та робота з файлами на мові С ++»

ХАІ.301. G3. 319а. номер 19 ЛР

Виконав студент гр. _____319а_____

_____Харченко Софія Янівна_____
(підпис, дата) (П.І.Б.)

Перевірив
_____Пявка Євгеній Валентинович_____
(підпис, дата) (П.І.Б.)

2026

## МЕТА РОБОТИ

Вивчити теоретичний матеріал по алгоритмам обробки масивів на мові С++, а також бібліотеки для роботи з файлами і реалізувати оголошення, введення з файлу, обробку і виведення в файл одновимірних і двовимірних масивів на мові С ++.

## ПОСТАНОВКА ЗАДАЧІ

Завдання 1. За допомогою текстового редактору створити текстовий файл «array_in_n.txt» з елементами вихідного масиву (n - номер варіанта). У програмі на С++ зчитати і перетворити цей масив відповідно до свого варіанту завдання (див. лаб.роб.№6, завд.2), ім'я файлу і необхідні змінні ввести з консолі. Вивести результати у файл «array_out_n.txt».

Завдання 2. За допомогою текстового редактору створити текстовий файл «matr_in_n.txt» з елементами вихідного двовимірного масиву (n - номер варіанта). У програмі зчитати і обробити матрицю відповідно до свого варіанту завдання (лаб.роб.№7, завд.1), ім'я файлу і необхідні змінні ввести з консолі. Дописати результати в той же файл.

Завдання 3. Вивчити метод сортування відповідно до свого варіанту (варіант 21), проаналізувати його складність і продемонструвати на прикладі з 7-ми елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої функції алгоритм сортування елементів масиву. Зчитування і виведення відсортованого масиву організувати на файлах.

## ВИКОНАННЯ РОБОТИ

Завдання 1.

Вирішення задачі Array 104

Вхідні дані (ім’я, опис, тип, обмеження):

filename – змінна, яка приймає постать назви файлу з якого зчитується розмір масиву та сам масив, тип даних: char (символи), обмеження: до 100 символів.

n – розмір вхідного масиву, тип даних: ціле число, обмеження: від 2 до 20.

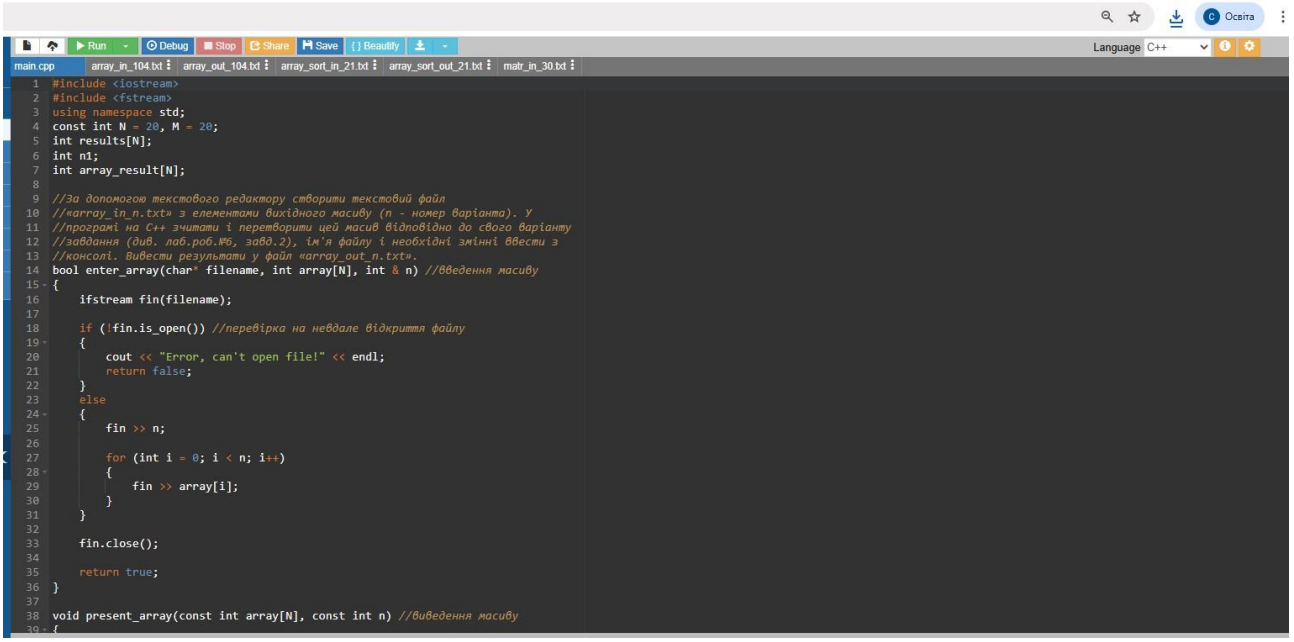
K – номер елемента з масиву після якого треба вставити M1 кількість нулів, тип даних: ціле число, обмеження: $1 \leq K \leq n$.

M1 – кількість нулів, які треба вставити після елемента масиву під номером K, тип даних: ціле число, обмеження: $1 \leq M1 \leq 10$.

Вихідні дані (ім’я, опис, тип):

array_result[] – масив, який зберігає модифікований вхідний масив, тип даних: ціле число.

Алгоритм вирішення показано нижче/на рис. 1, 2, 3, 4, 5, 6, 7 та діаграмі UML 1.



```
#include <iostream>
#include <fstream>
using namespace std;
const int N = 20, M = 20;
int results[N];
int n1;
int array_result[N];

//За допомогою текстового редактору створити текстовий файл
//«array_in_n.txt» з елементами вихідного масиву (n - номер варіанта). У
//програмі на C++ зчитати і перетворити цей масив відповідно до свого варіанту
//завдання (див. лаб.роб.№5, завд.2), ім'я файлу і необхідні змінні ввести з
//консолі. Вивести результати у файл «array_out_n.txt».
bool enter_array(char* filename, int array[N], int & n) //введення масиву
{
    ifstream fin(filename);

    if (!fin.is_open()) //перевірка на невдале відкриття файлу
    {
        cout << "Error, can't open file!" << endl;
        return false;
    }
    else
    {
        fin >> n;

        for (int i = 0; i < n; i++)
        {
            fin >> array[i];
        }
    }

    fin.close();

    return true;
}

void present_array(const int array[N], const int n) //виведення масиву
{
```

Рисунок 1 – Array 104

```cpp
{
    ifstream fin(filename);

    if (!fin.is_open()) //перевірка на невдале відкриття файлу
    {
        cout << "Error, can't open file!" << endl;
        return false;
    }
    else
    {
        fin >> n;

        for (int i = 0; i < n; i++)
        {
            fin >> array[i];
        }
    }

    fin.close();

    return true;
}

void present_array(const int array[N], const int n) //виведення масиву
{
    for (int i = 0; i < n; i++)
    {
        cout << array[i] << " ";
    }

    cout << endl;
}

void write_array(const int array[N], const int n) //записання модифікованого масиву у відповідний файл
{
    ofstream fout("array_out_104.txt");

    for (int i = 0; i < n1; i++)
    {
```

Рисунок 2 – Array 104

```cpp
}

void present_array(const int array[N], const int n) //виведення масиву
{
    for (int i = 0; i < n; i++)
    {
        cout << array[i] << " ";
    }

    cout << endl;
}

void write_array(const int array[N], const int n) //записання модифікованого масиву у відповідний файл
{
    ofstream fout("array_out_104.txt");

    for (int i = 0; i < n1; i++)
    {
        fout << array_result[i] << " ";
    }

    fout.close();
}

void do_task_1(int array[N], int & n, int K, int M1) //проведення мадифікації масиву
{
    int t = n - K;
    int array_1[t];
    int s = K;
    n1 = 0;

    for(int i = 0; i < K; i++)
    {
        cout << array[i] << " ";
        array_result[n1] = array[i];
        n1 = n1 + 1;
    }

    for(int j = 0; j < (n - K); j++)
```

Рисунок 3 – Array 104

```cpp
    ofstream fout("array_out_104.txt");

    for (int i = 0; i < n1; i++)
    {
        fout << array_result[i] << " ";
    }

    fout.close();
}

void do_task_1(int array[N], int & n, int K, int M1) //проведення мадифікації масиву
{
    int t = n - K;
    int array_1[t];
    int s = K;
    n1 = 0;

    for(int i = 0; i < K; i++)
    {
        cout << array[i] << " ";
        array_result[n1] = array[i];
        n1 = n1 + 1;
    }

    for(int j = 0; j < (n - K); j++)
    {
        array_1[j] = array[s];
        s = s + 1;
    }

    n1 = K;

    for(int l = K; l < (K + M1); l++)
    {
        array[l] = 0;
        cout << array[l] << " ";
        array_result[n1] = array[l];
        n1 = n1 + 1;
    }
```

Рисунок 4 – Array 104

```cpp
{
    int t = n - K;
    int array_1[t];
    int s = K;
    n1 = 0;

    for(int i = 0; i < K; i++)
    {
        cout << array[i] << " ";
        array_result[n1] = array[i];
        n1 = n1 + 1;
    }

    for(int j = 0; j < (n - K); j++)
    {
        array_1[j] = array[s];
        s = s + 1;
    }

    n1 = K;

    for(int l = K; l < (K + M1); l++)
    {
        array[l] = 0;
        cout << array[l] << " ";
        array_result[n1] = array[l];
        n1 = n1 + 1;
    }

    n1 = (K + M1);

    for(int y = 0; y < (n - K); y++)
    {
        cout << array_1[y] << " ";
        array_result[n1] = array_1[y];
        n1 = n1 + 1;
    }
```

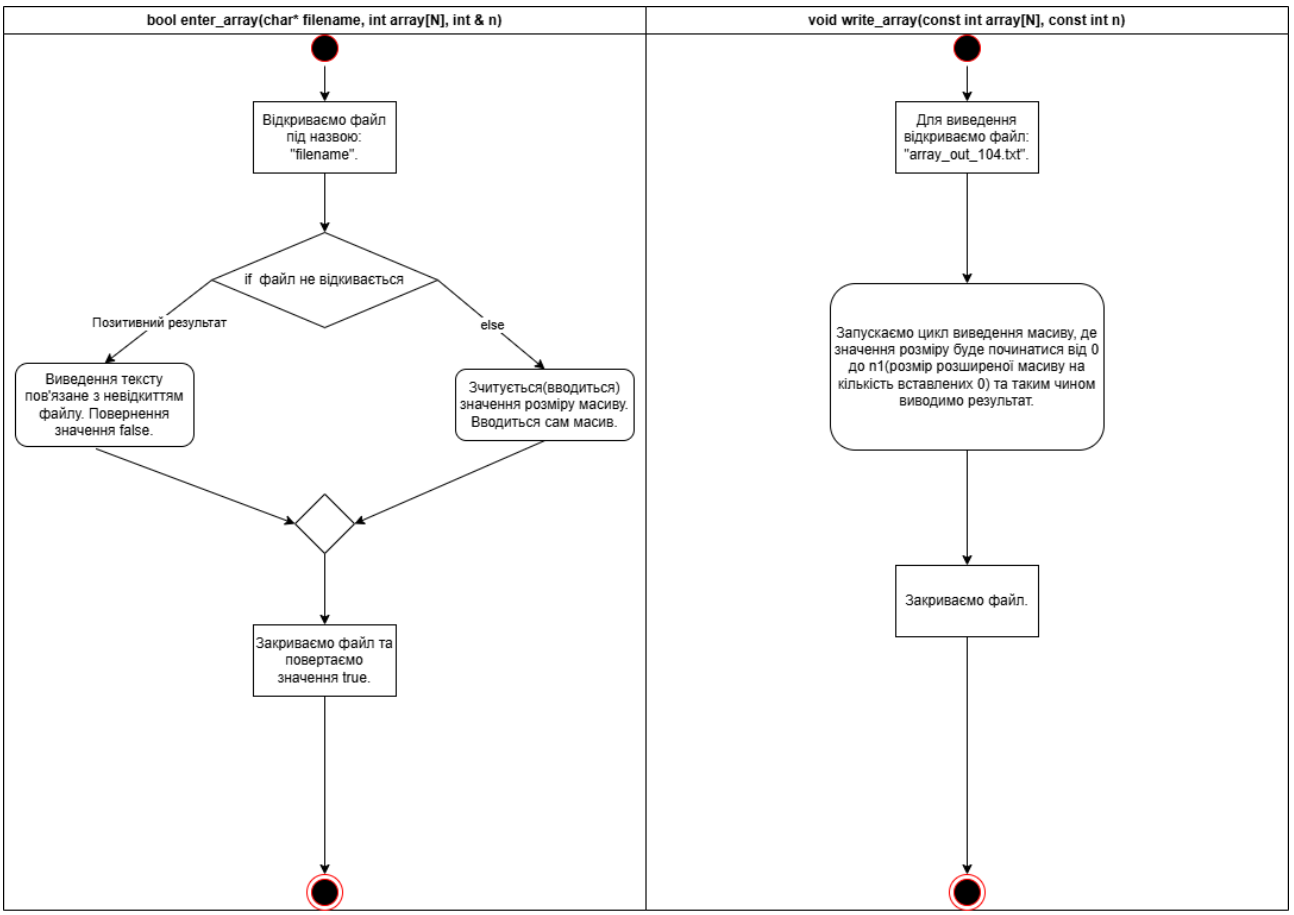
Рисунок 5 – Array 104

```
        n1 = n1 + 1;
    }

    for(int j = 0; j < (n - K); j++)
    {
        array_1[j] = array[s];
        s = s + 1;
    }

    n1 = K;

    for(int l = K; l < (K + M1); l++)
    {
        array[l] = 0;
        cout << array[l] << " ";
        array_result[n1] = array[l];
        n1 = n1 + 1;
    }

    n1 = (K + M1);

    for(int y = 0; y < (n - K); y++)
    {
        cout << array_1[y] << " ";
        array_result[n1] = array_1[y];
        n1 = n1 + 1;
    }

    cout << endl;
}

void task_1() //виконання завдання 1
{
    int array[N];
    int n;
    int K, M1;
    char filename[100];

    cout << "Enter filename: ";
```

Рисунок 6 – Array 104

```
        n1 = n1 + 1;
    }

    cout << endl;
}

void task_1() //виконання завдання 1
{
    int array[N];
    int n;
    int K, M1;
    char filename[100];

    cout << "Enter filename: ";
    cin.getline(filename, 100);

    if (enter_array(filename, array, n))
    {
        cout << "Array: ";
        present_array(array, n);

        cout << "Enter number of column after which zeros must be added: ";
        cin >> K;

        cout << "Enter needed amount of zeros to do a task: ";
        cin >> M1;

        n1 = n + M1;

        cout << "Modified array: ";
        do_task_1(array, n, K, M1);
        write_array(array, n);
    }

    cout << endl;
}

//За допомогою текстового редактору створити текстовий файл
```

Рисунок 7 – Array 104




Діаграма UML 1 – Array 104

Лістинг коду вирішення задачі Array 104 наведено в дод. А (стор. 17 - 19). Екран роботи програми показаний на рис. Б.1.1 та Б.1.2.

Завдання 2.

Вирішення задачі Matrix 30

Вхідні дані (ім’я, опис, тип, обмеження):

filename – змінна, яка приймає постать назви файлу з якого зчитується розмір масиву та сам масив, тип даних: char (символи), обмеження: до 100 символів.

row – кількість рядків в матриці, тип даних: ціле число, обмеження: від 2 до 20.

col – кількість стовпців в матриці, тип даних: ціле число, обмеження: від 2 до 20.

Вихідні дані (ім’я, опис, тип):

results[] – масив, який зберігає результати виконання завдання, тип даних: ціле число.

Алгоритм вирішення показано нижче/на рис. 8, 9, 10, 11, 12, 13, 14, 15, 16 та діаграмі UML 2.

```cpp
}

//За допомогою текстового редактору створити текстовий файл
//«matr_in_n.txt» з елементами вихідного двовимірного масиву (n - номер
//варіанта). У програмі зчитати і обробити матрицю відповідно до свого
//варіанту завдання (лаб.роб.№7, завд.1), ім'я файлу і необхідні змінні ввести з
//консолі. Дописати результати в той же файл.
bool enter_matrix(char* filename, int matrix[M][N], int & m, int & n) //введення матриці
{
    ifstream fin(filename);

    if (!fin.is_open()) //перевірка на невдале відкриття файлу
    {
        cout << "Error, can't open file!" << endl;
        return false;
    }
    else
    {
        fin >> m;
        fin >> n;

        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                fin >> matrix[i][j];
            }
        }
    }

    fin.close();

    return true;
}

void present_matrix(const int matrix[M][N], const int m, const int n) //виведення матриці
{
    for (int i = 0; i < m; i++)
```

Рисунок 8 – Matrix 30

```cpp
    {
        fin >> m;
        fin >> n;

        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                fin >> matrix[i][j];
            }
        }
    }

    fin.close();

    return true;
}

void present_matrix(const int matrix[M][N], const int m, const int n) //виведення матриці
{
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << matrix[i][j] << " ";
        }

        cout << endl;
    }
}

void write_matrix(char* filename, const int matrix[M][N], const int m, const int n) //записання результату у відповідний файл
{
    ofstream fout("matr_in_30.txt", ios_base::app);

    fout << " \nResult: ";

    for (int l = 0; l < n; l++)
    {
```

Рисунок 9 – Matrix 30

```
    return true;
}

void present_matrix(const int matrix[M][N], const int m, const int n) //виведення матриці
{
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << matrix[i][j] << " ";
        }

        cout << endl;
    }
}

void write_matrix(char* filename, const int matrix[M][N], const int m, const int n) //записання результату у відповідний файл
{
    ofstream fout("matr_in_30.txt", ios_base::app);

    fout << " \nResult: ";

    for (int l = 0; l < n; l++)
    {
        fout << results[l] << " ";
    }

    fout.close();
}

void do_task_2(int matrix[M][N], int & m, int & n) //виконання підрахунків, скільки елементів з колони є більших за її середню арифметичну
{
    int a, k, s, sum, count_amount, count_elements, constant;
    float arith_mean = 0;

    constant = 0;
    k = 0;
    s = 0;
```

Рисунок 10 – Matrix 30

```
}

void write_matrix(char* filename, const int matrix[M][N], const int m, const int n) //записання результату у відповідний файл
{
    ofstream fout("matr_in_30.txt", ios_base::app);

    fout << " \nResult: ";

    for (int l = 0; l < n; l++)
    {
        fout << results[l] << " ";
    }

    fout.close();
}

void do_task_2(int matrix[M][N], int & m, int & n) //виконання підрахунків, скільки елементів з колони є більших за її середню арифметичну
{
    int a, k, s, sum, count_amount, count_elements, constant;
    float arith_mean = 0;

    constant = 0;
    k = 0;
    s = 0;
    sum = 0;
    count_amount = 0;
    count_elements = 0;
    a = 0;

    while (k < n)
    {
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (constant == j)
                {
                    s = matrix[i][j];
```

Рисунок 11 – Matrix 30

```
}

void do_task_2(int matrix[M][N], int & m, int & n) //Виконання підрахунків, скільки елементів з колони є більших за її середню арифметичну
{
    int a, k, s, sum, count_amount, count_elements, constant;
    float arith_mean = 0;

    constant = 0;
    k = 0;
    s = 0;
    sum = 0;
    count_amount = 0;
    count_elements = 0;
    a = 0;

    while (k < n)
    {
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (constant == j)
                {
                    s = matrix[i][j];
                    sum = sum + s;
                    count_amount += 1;
                }
            }
        }

        arith_mean = (sum / count_amount);

        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (constant == j)
                {
```

Рисунок 12 – Matrix 30

```
    sum = 0;
    count_amount = 0;
    count_elements = 0;
    a = 0;

    while (k < n)
    {
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (constant == j)
                {
                    s = matrix[i][j];
                    sum = sum + s;
                    count_amount += 1;
                }
            }
        }

        arith_mean = (sum / count_amount);

        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (constant == j)
                {
                    if (matrix[i][j] > arith_mean)
                    {
                        count_elements += 1;
                    }

                    if (i == (m - 1))
                    {
                        a = count_elements;
                        results[j] = a;
                    }
                }
```

Рисунок 13 – Matrix 30

```
                if (constant == j)
                {
                    s = matrix[i][j];
                    sum = sum + s;
                    count_amount += 1;
                }
            }
        }

        arith_mean = (sum / count_amount);

        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (constant == j)
                {
                    if (matrix[i][j] > arith_mean)
                    {
                        count_elements += 1;
                    }

                    if (i == (m - 1))
                    {
                        a = count_elements;
                        results[j] = a;
                    }
                }
            }
        }

        cout << "Amount of elements that meet the task conditions in column: "<< count_elements << endl;



        constant = constant + 1;
        k = k + 1;
```

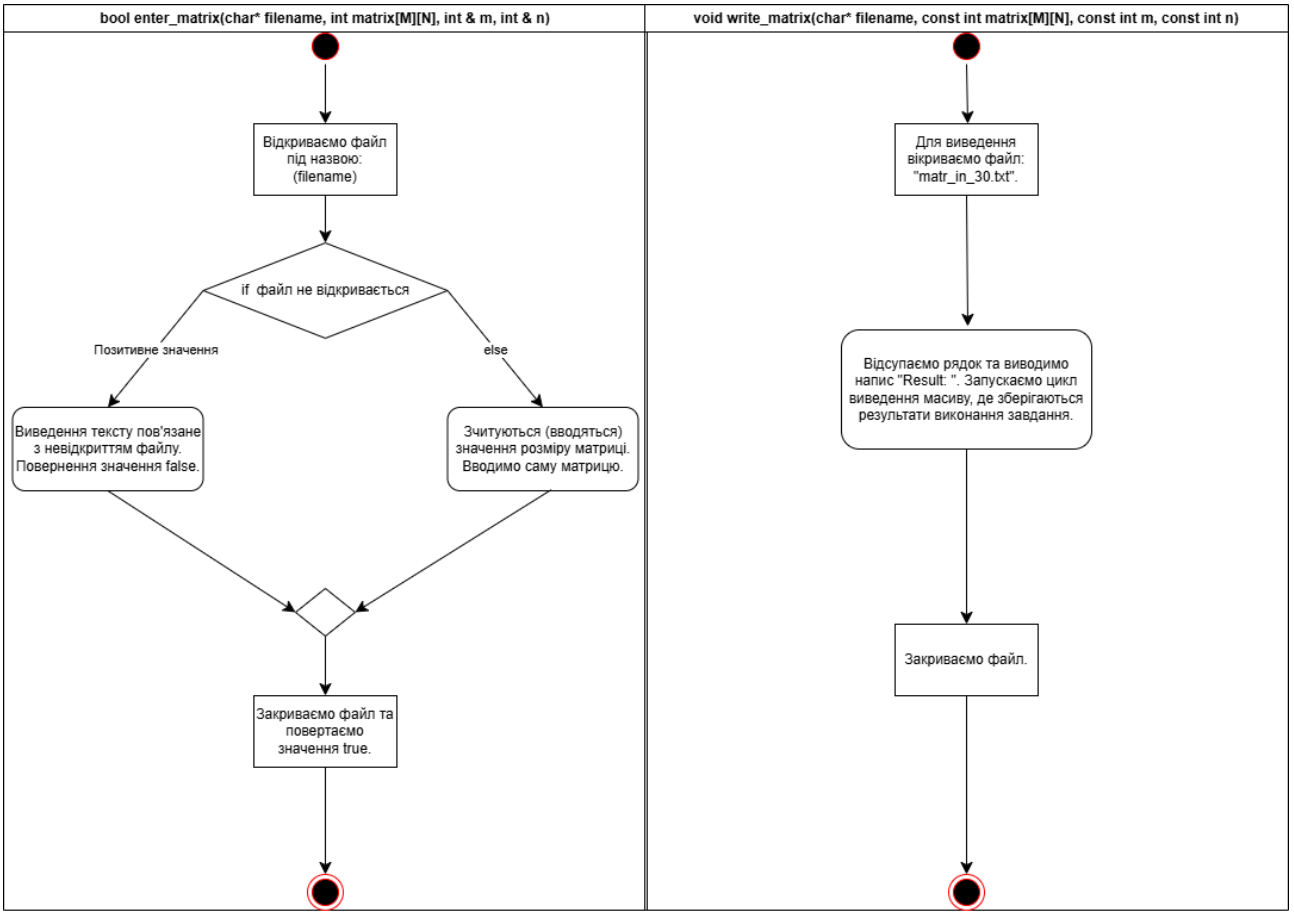
Рисунок 14 – Matrix 30

```
                    if (matrix[i][j] > arith_mean)
                    {
                        count_elements += 1;
                    }

                    if (i == (m - 1))
                    {
                        a = count_elements;
                        results[j] = a;
                    }
                }
            }
        }

        cout << "Amount of elements that meet the task conditions in column: "<< count_elements << endl;



        constant = constant + 1;
        k = k + 1;

        a = 0;
        sum = 0;
        count_amount = 0;
        count_elements = 0;
    }

    cout << endl;
}

void task_2() //виконання завдання 2
{
    int matrix[M][N];
    int row, col;
    char filename[100];

    cout << "Enter filename: ";
    cin.getline(filename, 100);
```

Рисунок 15 – Matrix 30

```
        a = 0;
        sum = 0;
        count_amount = 0;
        count_elements = 0;
    }

    cout << endl;
}

void task_2() //виконання завдання 2
{
    int matrix[M][N];
    int row, col;
    char filename[100];

    cout << "Enter filename: ";
    cin.getline(filename, 100);

    if (enter_matrix(filename, matrix, row, col))
    {
        cout << "Matrix: " << endl;
        present_matrix(matrix, row, col);

        do_task_2(matrix, row, col);
        write_matrix(filename, matrix, row, col);
    }
}

//Вивчити метод сортування відповідно до свого варіанту (варіант 21)
//проаналізувати його складність і продемонструвати на прикладі з 7-ми
//елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої функції
//алгоритм сортування елементів масиву. Зчитування і виведення відсортованого
//масиву організувати на файлах.
bool enter_array_sort(char* filename, int array_sort[N], int & n) //введення масиву для сортування
{
    ifstream fin(filename);

    if (!fin.is_open()) //перевірка на невдале відкриття файлу
    {
```

Рисунок 16 – Matrix 30




Діаграма UML 2 – Matrix 30

Лістинг коду вирішення задачі Matrix 30 наведено в дод. А (стор. 19 -21). Екран роботи програми показаний на рис. Б.2.1 та Б.2.2.

Завдання 3.

Вирішення задачі Сортування 21

Вхідні дані (ім’я, опис, тип, обмеження):

filename – змінна, яка приймає постать назви файлу з якого зчитується розмір масиву та сам масив, типу char (символи), обмеження: до 100 символів.

n – розмір вхідного масиву, тип даних: ціле число, обмеження: від 2 до 20.

Вихідні дані (ім’я, опис, тип):

array_sort[] – масив, який зберігає посортований вхідний масив, тип даних: ціле число.

Алгоритм вирішення показано нижче/на рис. 17, 18, 19, 20, 21 та діаграмі UML 3.

```
    }
}

//Вивчити метод сортування відповідно до свого варіанту (варіант 21)
//проаналізувати його складність і продемонструвати на прикладі з 7-ми
//елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої функції
//алгоритм сортування елементів масиву. Зчитування і виведення відсортованого
//масиву організувати на файлах.
bool enter_array_sort(char* filename, int array_sort[N], int & n) //Введення масиву для сортування
{
    ifstream fin(filename);

    if (!fin.is_open()) //перевірка на невдале відкриття файлу
    {
        cout << "Error, can't open file!" << endl;
        return false;
    }
    else
    {
        fin >> n;

        for (int i = 0; i < n; i++)
        {
            fin >> array_sort[i];
        }
    }

    fin.close();

    return true;
}

void present_array_sort(const int array_sort[N], const int n) //Виведення масуву для сортування
{
    for (int i = 0; i < n; i++)
    {
        cout << array_sort[i] << " ";
    }
```

Рисунок 17 – Сортування 21

```
        for (int i = 0; i < n; i++)
        {
            fin >> array_sort[i];
        }
    }

    fin.close();

    return true;
}

void present_array_sort(const int array_sort[N], const int n) //виведення масуву для сортування
{
    for (int i = 0; i < n; i++)
    {
        cout << array_sort[i] << " ";
    }

    cout << endl;
}

void write_array_sort(const int array_sort[N], const int n) //записання відсортованого масиву у відповідний файл
{
    ofstream fout("array_sort_out_21.txt");

    for (int i = 0; i < n; i++)
    {
        fout << array_sort[i] << " ";
    }

    fout.close();
}

void do_task_3(int array_sort[N], int & n) //виконання сортування масиву
{
    int a, b, k;

    k = 0;
```

Рисунок 18 – Сортування 21

```
    cout << endl;
}

void write_array_sort(const int array_sort[N], const int n) //записання відсортованого масиву у відповідний файл
{
    ofstream fout("array_sort_out_21.txt");

    for (int i = 0; i < n; i++)
    {
        fout << array_sort[i] << " ";
    }

    fout.close();
}

void do_task_3(int array_sort[N], int & n) //виконання сортування масиву
{
    int a, b, k;

    k = 0;
    a = 0;

    for (int i = 0; i < n; i++)
    {
        a = array_sort[i];
        k = i;

        while ((k > 0) && (array_sort[k - 1] > a))
        {
            b = array_sort[k];
            array_sort[k] = array_sort[k - 1];
            array_sort[k - 1] = b;

            k--;
        }
    }

    array_sort[k] = a;
```

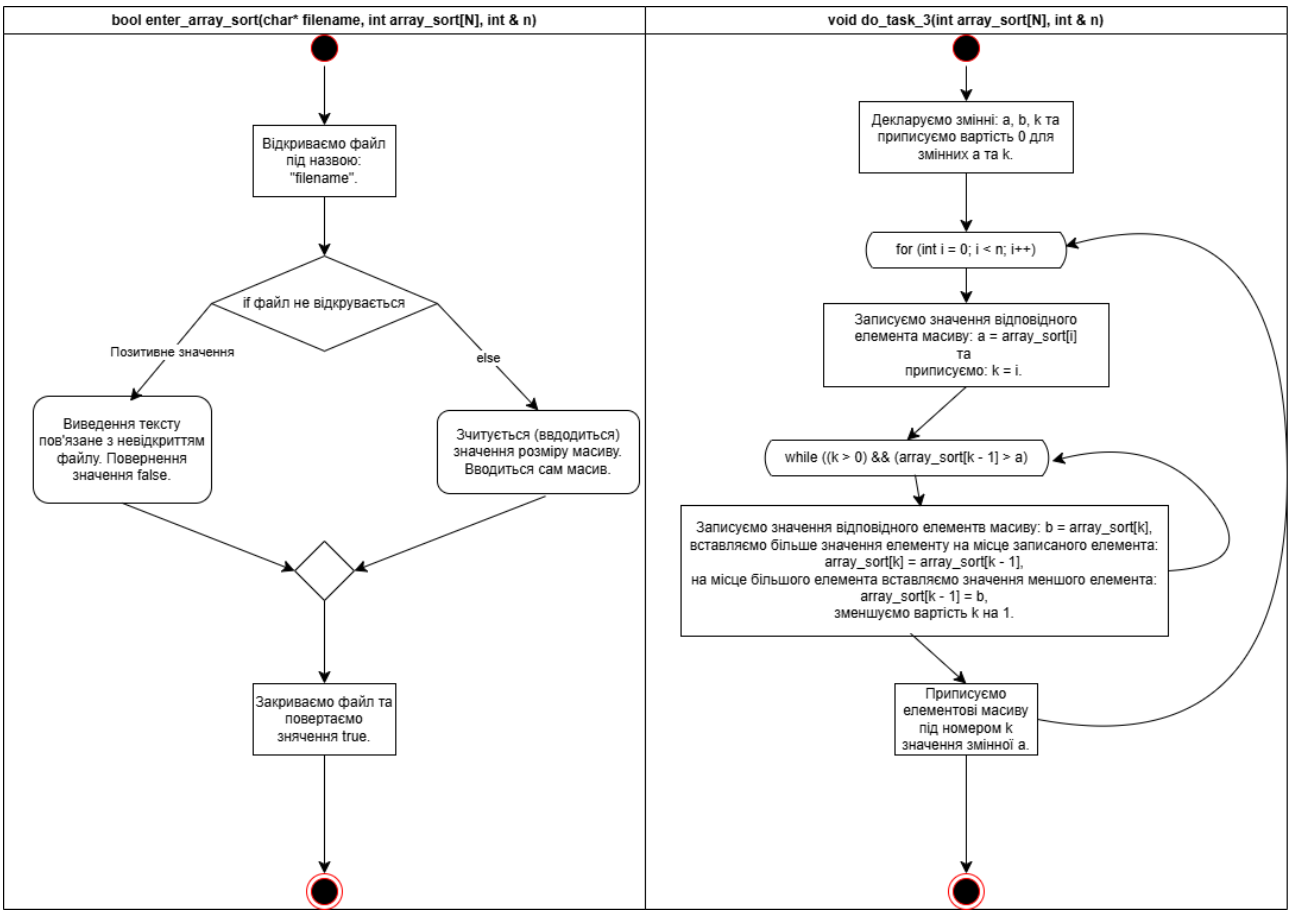
Рисунок 19 – Сортування 21

```cpp
    }

    fout.close();
}

void do_task_3(int array_sort[N], int & n) //виконання сортування масиву
{
    int a, b, k;

    k = 0;
    a = 0;

    for (int i = 0; i < n; i++)
    {
        a = array_sort[i];
        k = i;

        while ((k > 0) && (array_sort[k - 1] > a))
        {
            b = array_sort[k];
            array_sort[k] = array_sort[k - 1];
            array_sort[k - 1] = b;

            k--;
        }

        array_sort[k] = a;
    }
}

void task_3() //виконання завдання 3
{
    int array_sort[N];
    int n;
    char filename[100];

    cout << "Enter filename: ";
    cin.getline (filename, 100);
```

Рисунок 20 – Сортування 21

```cpp
        while ((k > 0) && (array_sort[k - 1] > a))
        {
            b = array_sort[k];
            array_sort[k] = array_sort[k - 1];
            array_sort[k - 1] = b;

            k--;
        }

        array_sort[k] = a;
    }
}

void task_3() //виконання завдання 3
{
    int array_sort[N];
    int n;
    char filename[100];

    cout << "Enter filename: ";
    cin.getline (filename, 100);

    if (enter_array_sort(filename, array_sort, n))
    {
        cout << "Array to sort: ";
        present_array_sort(array_sort, n);

        cout << "Sorted array: ";
        do_task_3(array_sort, n);
        present_array_sort(array_sort, n);
        write_array_sort(array_sort, n);
    }

    cout << endl;
}

int main() //головне меню
{
    int task_num;
```

Рисунок 21 – Сортування 21




Діаграма UML 3 – Сортування 21

Лістинг коду вирішення задачі Сортування 21 наведено в дод. А (стор. 21 - 23).

Екран роботи програми показаний на рис. Б.3.1 та Б.3.2.

## ВИСНОВКИ

Під час виконання лабораторної роботи було засвоєно вміння використання файлів в програмуванні. На певних етапах виникали складнощі, але остаточно всі вони були розв’язані.

# ДОДАТОК А

## Лістинг коду програми

```
#include <iostream>
#include <fstream>
using namespace std;
const int N = 20, M = 20;
int results[N];
int n1;
int array_result[N];

//За допомогою текстового редактору створити текстовий файл
//«array_in_n.txt» з елементами вихідного масиву (n - номер варіанта). У
//програмі на С++ зчитати і перетворити цей масив відповідно до свого
варіанту
//завдання (див. лаб.роб.№6, завд.2), ім'я файлу і необхідні змінні ввести з
//консолі. Вивести результати у файл «array_out_n.txt».
bool enter_array(char* filename, int array[N], int & n) //введення масиву
{
    ifstream fin(filename);

    if (!fin.is_open()) //перевірка на невдале відкриття файлу
    {
        cout << "Error, can't open file!" << endl;
        return false;
    }
    else
    {
        fin >> n;

        for (int i = 0; i < n; i++)
        {
            fin >> array[i];
        }
    }

    fin.close();

    return true;
}

void present_array(const int array[N], const int n) //виведення масиву
{
    for (int i = 0; i < n; i++)
    {
        cout << array[i] << " ";
    }

    cout << endl;
}

void write_array(const int array[N], const int n) //записання модифікованого
масиву у відповідний файл
{
    ofstream fout("array_out_104.txt");
```

```
    for (int i = 0; i < n1; i++)
    {
        fout << array_result[i] << " ";
    }

    fout.close();
}

void do_task_1(int array[N], int & n, int K, int M1) //проведення
мадифікації масиву
{
    int t = n - K;
    int array_1[t];
    int s = K;
    n1 = 0;

    for(int i = 0; i < K; i++)
    {
        cout << array[i] << " ";
        array_result[n1] = array[i];
        n1 = n1 + 1;
    }

    for(int j = 0; j < (n - K); j++)
    {
        array_1[j] = array[s];
        s = s + 1;
    }

    n1 = K;

    for(int l = K; l < (K + M1); l++)
    {
        array[l] = 0;
        cout << array[l] << " ";
        array_result[n1] = array[l];
        n1 = n1 + 1;
    }

    n1 = (K + M1);

    for(int y = 0; y < (n - K); y++)
    {
        cout << array_1[y] << " ";
        array_result[n1] = array_1[y];
        n1 = n1 + 1;
    }

    cout << endl;
}

void task_1() //виконання завдання 1
{
    int array[N];
    int n;
    int K, M1;
    char filename[100];

    cout << "Enter filename: ";
```

```
    cin.getline(filename, 100);

    if (enter_array(filename, array, n))
    {
        cout << "Array: ";
        present_array(array, n);

        cout << "Enter number of column after which zeros must be added: ";
        cin >> K;

        cout << "Enter needed amount of zeros to do a task: ";
        cin >> M1;

        n1 = n + M1;

        cout << "Modified array: ";
        do_task_1(array, n, K, M1);
        write_array(array, n);
    }

    cout << endl;
}

//За допомогою текстового редактору створити текстовий файл
//«matr_in_n.txt» з елементами вихідного двовимірного масиву (n - номер
//варіанта). У програмі зчитати і обробити матрицю відповідно до свого
//варіанту завдання (лаб.роб.№7, завд.1), ім'я файлу і необхідні змінні
ввести з
//консолі. Дописати результати в той же файл.
bool enter_matrix(char* filename, int matrix[M][N], int & m, int & n)
//введення матриці
{
    ifstream fin(filename);

    if (!fin.is_open()) //перевірка на невдале відкриття файлу
    {
        cout << "Error, can't open file!" << endl;
        return false;
    }
    else
    {
        fin >> m;
        fin >> n;

        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                fin >> matrix[i][j];
            }
        }
    }

    fin.close();

    return true;
}
```

```
void present_matrix(const int matrix[M][N], const int m, const int n)
//виведення матриці
{
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            cout << matrix[i][j] << " ";
        }

        cout << endl;
    }
}

void write_matrix(char* filename, const int matrix[M][N], const int m, const
int n) //записання результату у відповідний файл
{
    ofstream fout("matr_in_30.txt", ios_base::app);

    fout << " \nResult: ";

    for (int l = 0; l < n; l++)
    {
        fout << results[l] << " ";
    }

    fout.close();
}

void do_task_2(int matrix[M][N], int & m, int & n) //виконання підрахунків,
скільки елементів з колони є більших за її середню арифметичну
{
    int a, k, s, sum, count_amount, count_elements, constant;
    float arith_mean = 0;

    constant = 0;
    k = 0;
    s = 0;
    sum = 0;
    count_amount = 0;
    count_elements = 0;
    a = 0;

    while (k < n)
    {
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (constant == j)
                {
                    s = matrix[i][j];
                    sum = sum + s;
                    count_amount += 1;
                }
            }
        }

        arith_mean = (sum / count_amount);
```

```
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < n; j++)
            {
                if (constant == j)
                {
                    if (matrix[i][j] > arith_mean)
                    {
                        count_elements += 1;
                    }

                    if (i == (m - 1))
                    {
                        a = count_elements;
                        results[j] = a;
                    }
                }
            }
        }

        cout << "Amount of elements that meet the task conditions in column:
"<< count_elements << endl;



        constant = constant + 1;
        k = k + 1;

        a = 0;
        sum = 0;
        count_amount = 0;
        count_elements = 0;
    }

    cout << endl;
}

void task_2() //виконання завдання 2
{
    int matrix[M][N];
    int row, col;
    char filename[100];

    cout << "Enter filename: ";
    cin.getline(filename, 100);

    if (enter_matrix(filename, matrix, row, col))
    {
        cout << "Matrix: " << endl;
        present_matrix(matrix, row, col);

        do_task_2(matrix, row, col);
        write_matrix(filename, matrix, row, col);
    }
}

//Вивчити метод сортування відповідно до свого варіанту (варіант 21)
//проаналізувати його складність і продемонструвати на прикладі з 7-ми
```

```
//елементів (відповідно до свого варіанту). Реалізувати у вигляді окремої
функції
//алгоритм сортування елементів масиву. Зчитування і виведення
відсортованого
//масиву організувати на файлах.
bool enter_array_sort(char* filename, int array_sort[N], int & n) //введення
масиву для сортування
{
    ifstream fin(filename);

    if (!fin.is_open()) //перевірка на невдале відкриття файлу
    {
        cout << "Error, can't open file!" << endl;
        return false;
    }
    else
    {
        fin >> n;

        for (int i = 0; i < n; i++)
        {
            fin >> array_sort[i];
        }
    }

    fin.close();

    return true;
}

void present_array_sort(const int array_sort[N], const int n) //виведення
масуву для сортування
{
    for (int i = 0; i < n; i++)
    {
        cout << array_sort[i] << " ";
    }

    cout << endl;
}

void write_array_sort(const int array_sort[N], const int n) //записання
відсортованого масиву у відповідний файл
{
    ofstream fout("array_sort_out_21.txt");

    for (int i = 0; i < n; i++)
    {
        fout << array_sort[i] << " ";
    }

    fout.close();
}

void do_task_3(int array_sort[N], int & n) //виконання сортування масиву
{
    int a, b, k;

    k = 0;
```

```
    a = 0;

    for (int i = 0; i < n; i++)
    {
        a = array_sort[i];
        k = i;

        while ((k > 0) && (array_sort[k - 1] > a))
        {
            b = array_sort[k];
            array_sort[k] = array_sort[k - 1];
            array_sort[k - 1] = b;

            k--;
        }

        array_sort[k] = a;
    }
}

void task_3() //виконання завдання 3
{
    int array_sort[N];
    int n;
    char filename[100];

    cout << "Enter filename: ";
    cin.getline (filename, 100);

    if (enter_array_sort(filename, array_sort, n))
    {
        cout << "Array to sort: ";
        present_array_sort(array_sort, n);

        cout << "Sorted array: ";
        do_task_3(array_sort, n);
        present_array_sort(array_sort, n);
        write_array_sort(array_sort, n);
    }

    cout << endl;
}

int main() //головне меню
{
    int task_num;

    do
    {
        cout << "Task number: ";
        cin >> task_num;
        cin.ignore(32767, '\n');

        switch (task_num)
        {
            case 1: task_1(); break;
            case 2: task_2(); break;
            case 3: task_3(); break;
            case 0: cout << "Ending of doing tasks."; break;
```

```
            default: cout << "Wrong task number!" << endl;
        }
    }
    while (task_num != 0);
}
```

# ДОДАТОК Б

## Скрін-шоти вікна виконання програми

```
Task number: 1
Enter filename: array_in_104.txt
Array: 7 5 4 8 9 12 11 14 18 27
Enter number of column after which zeros must be added: 4
Enter needed amount of zeros to do a task: 5
Modified array: 7 5 4 8 0 0 0 0 0 9 12 11 14 18 27
```

Рисунок Б.1.1 – Екран виконання програми для вирішення завдання Array 104

```
main.cpp   array_in_104.txt   array_out_104.txt   array_sort_in_21.txt   array_sort_out_21.txt   matr_in_30.txt
1  7 5 4 8 0 0 0 0 0 9 12 11 14 18 27
```

Рисунок Б.1.2 – Екран виконання програми для вирішення завдання Array 104

```
Task number: 2
Enter filename: matr_in_30.txt
Matrix:
5 1 3 1 7 1
6 2 3 2 7 2
Amount of elements that meet the task conditions in column: 1
Amount of elements that meet the task conditions in column: 1
Amount of elements that meet the task conditions in column: 0
Amount of elements that meet the task conditions in column: 1
Amount of elements that meet the task conditions in column: 0
Amount of elements that meet the task conditions in column: 1
```

Рисунок Б.2.1 – Екран виконання програми для вирішення завдання Matrix 30

Рисунок Б.2.2 – Екран виконання програми для вирішення завдання Matrix 30

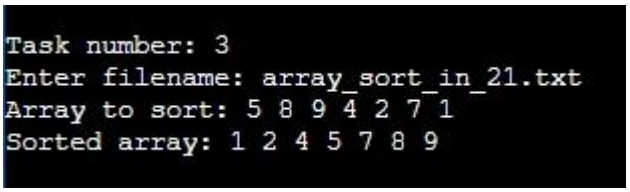



Рисунок Б.3.1 – Екран виконання програми для вирішення завдання Сортування 21

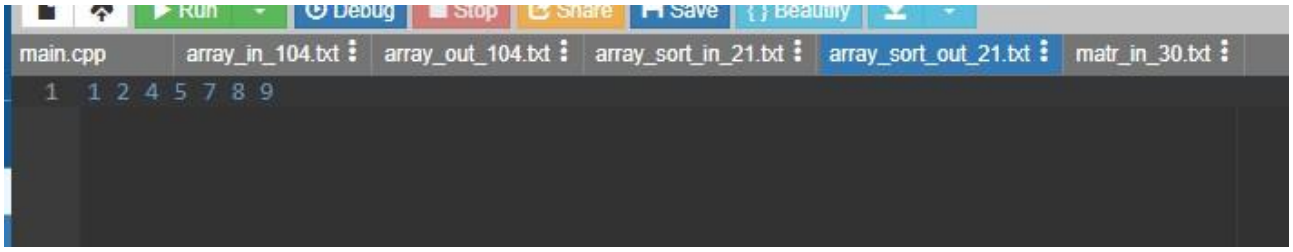
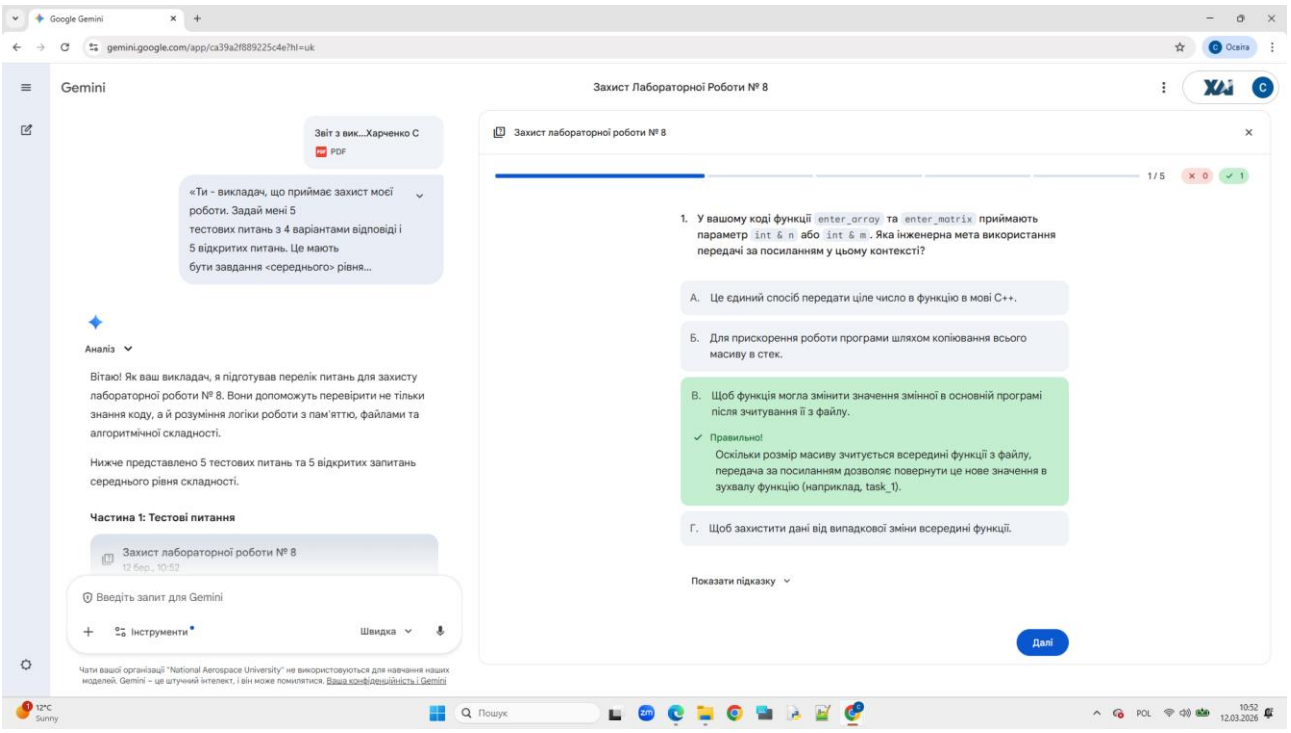
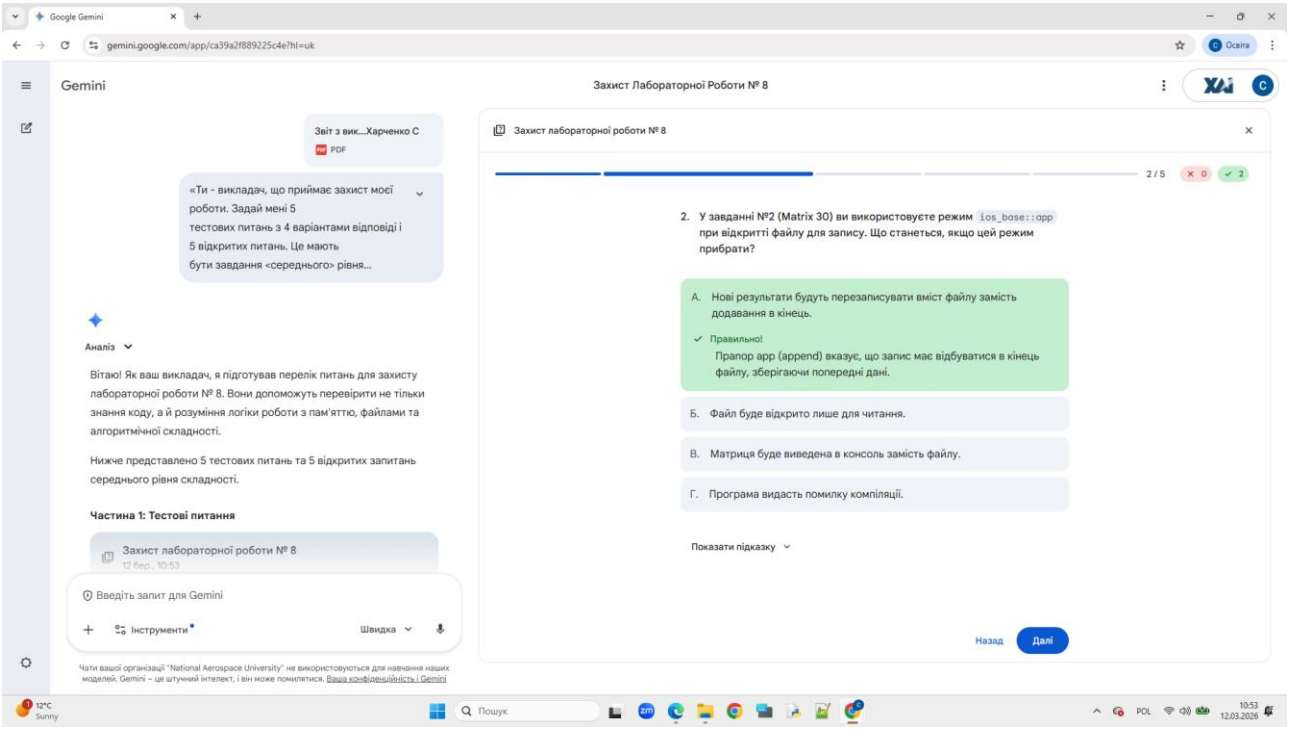
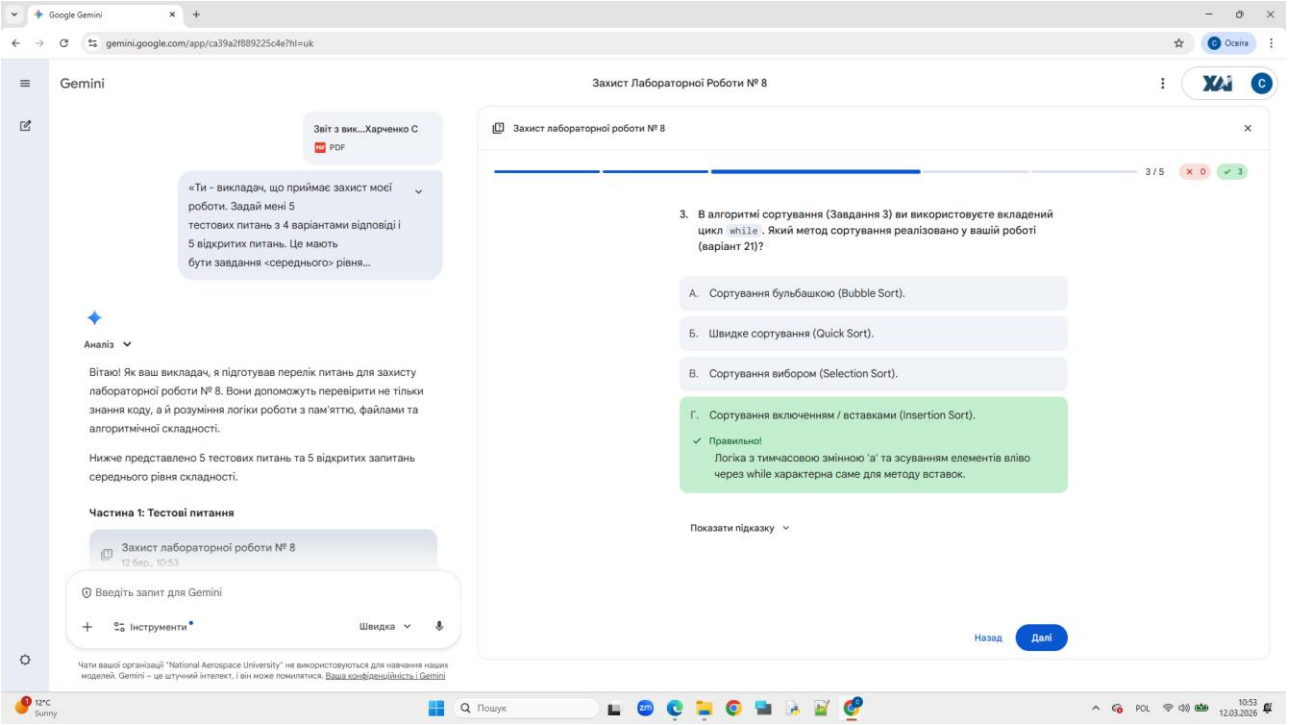


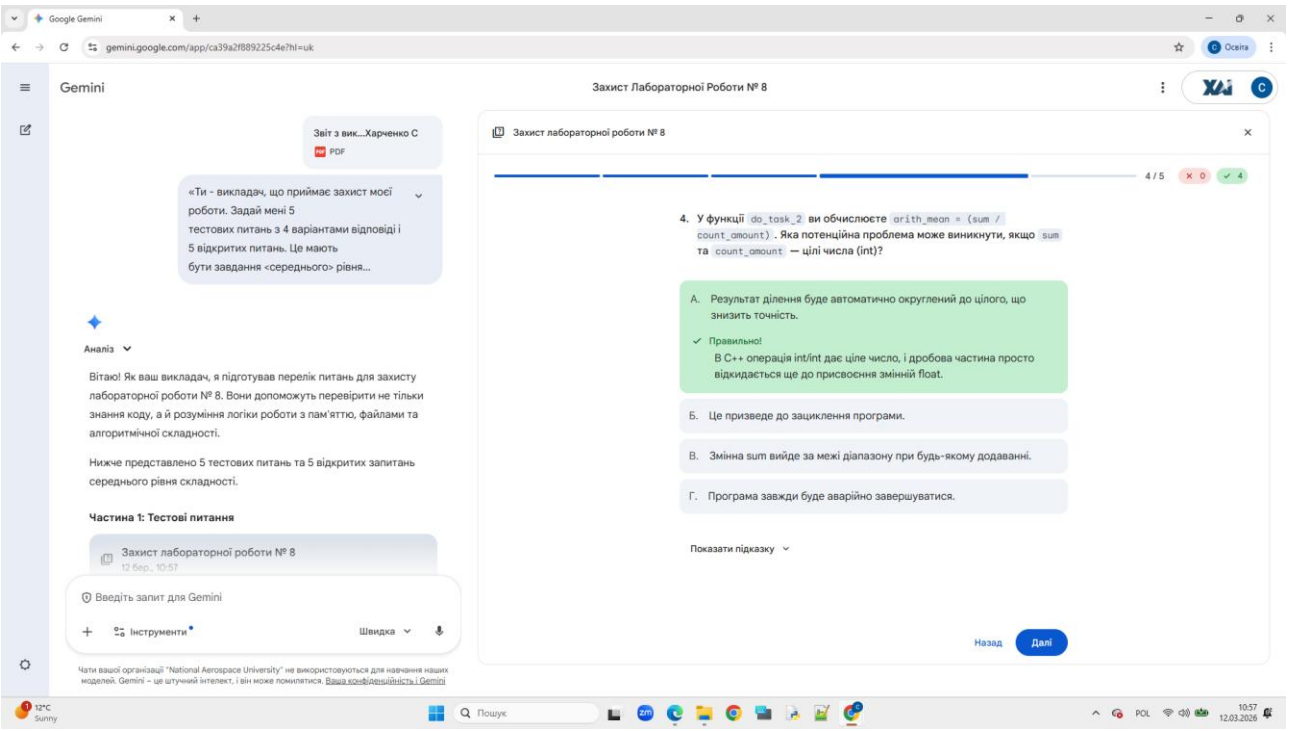

Рисунок Б.3.2 – Екран виконання програми для вирішення завдання Сортування 21

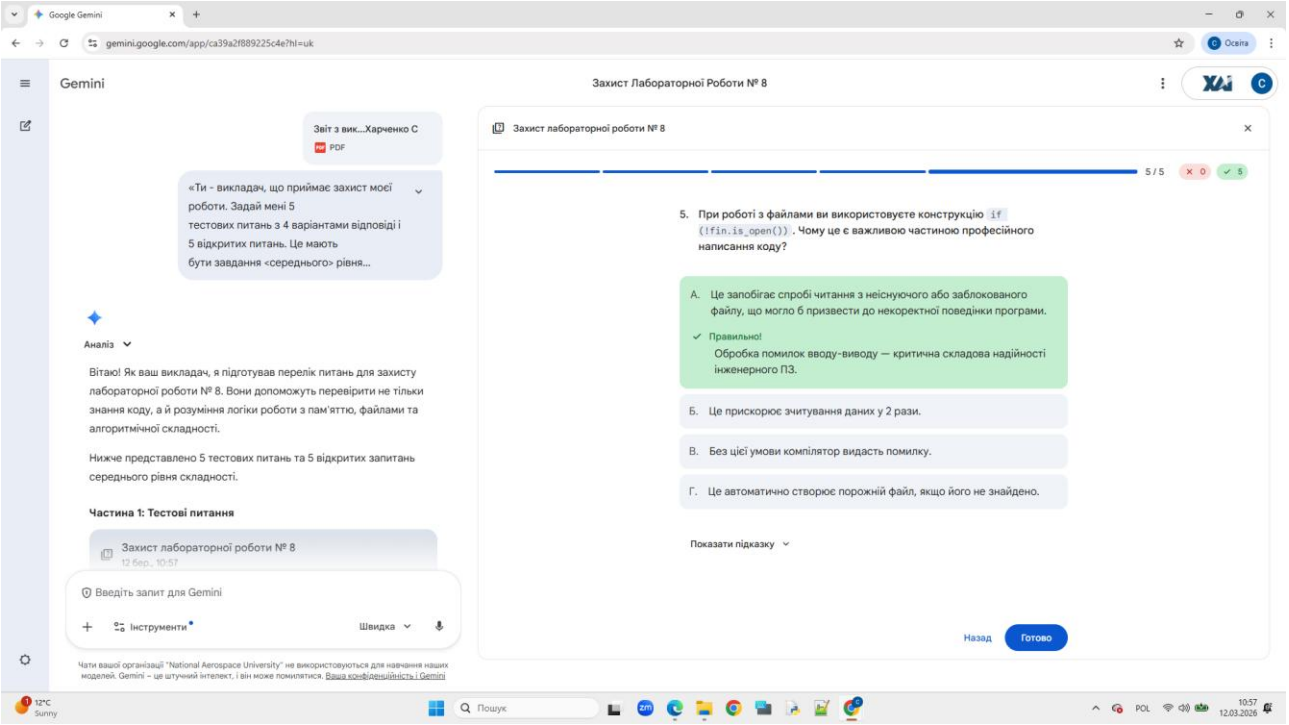
# ДОДАТОК В
## Діалог з ШІ для самоаналізу

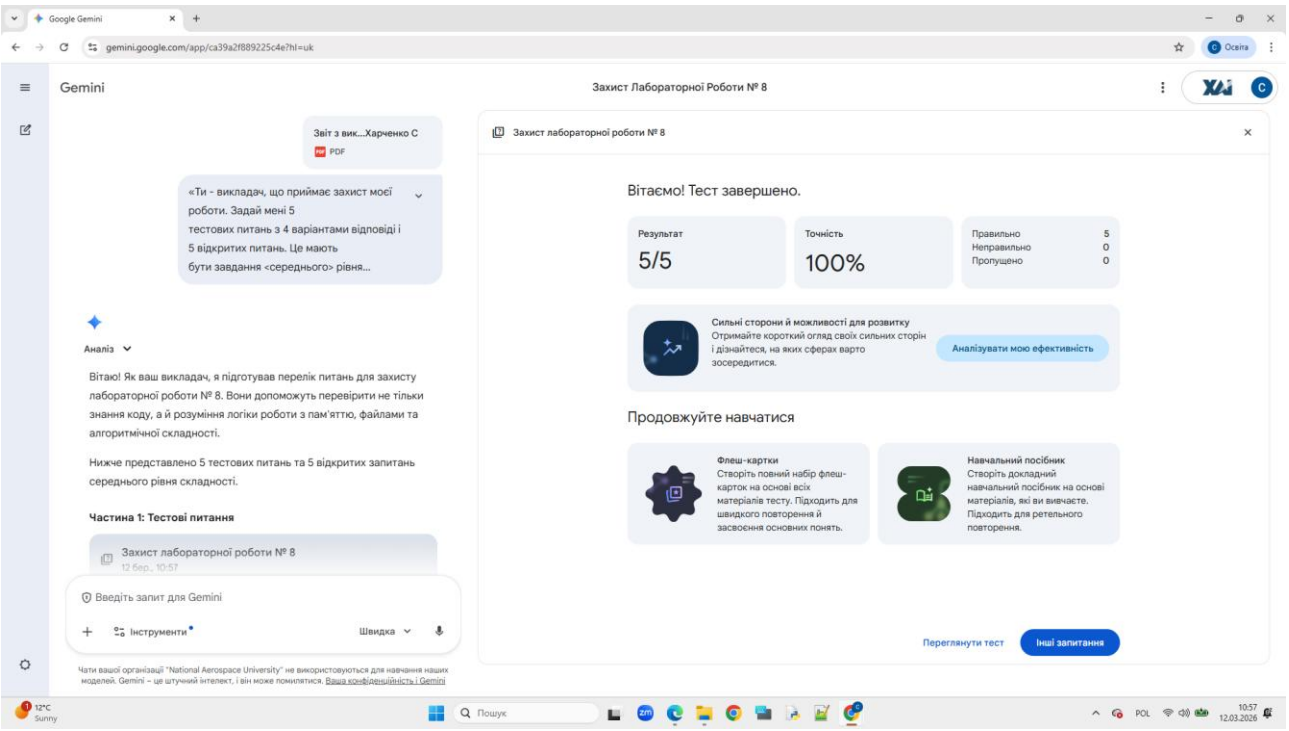



Google Gemini
gemini.google.com/app/ca39a2f889225c4e?hl=uk
Gemini
Захист Лабораторної Роботи № 8
Звіт з вик...Харченко С
PDF
«Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня...
Аналіз
Вітаю! Як ваш викладач, я підготував перелік питань для захисту лабораторної роботи № 8. Вони допоможуть перевірити не тільки знання коду, а й розуміння логіки роботи з пам'яттю, файлами та алгоритмічної складності.
Нижче представлено 5 тестових питань та 5 відкритих запитань середнього рівня складності.
Частина 1: Тестові питання
Захист лабораторної роботи № 8
12 бер., 10:53
Введіть запит для Gemini
Інструменти
Швидка
Чати вашої організації "National Aerospace University" не використовуються для навчання наших моделей. Gemini - це штучний інтелект, і він може помилятися. Ваша конфіденційність і Gemini
Захист лабораторної роботи № 8
3 / 5
3. В алгоритмі сортування (Завдання 3) ви використовуєте вкладений цикл while. Який метод сортування реалізовано у вашій роботі (варіант 21)?
А. Сортування бульбашкою (Bubble Sort).
Б. Швидке сортування (Quick Sort).
В. Сортування вибором (Selection Sort).
Г. Сортування включенням / вставками (Insertion Sort).
Правильно!
Логіка з тимчасовою змінною 'a' та зсуванням елементів вліво через while характерна саме для методу вставок.
Показати підказку
Назад
Далі


Google Gemini
gemini.google.com/app/ca39a2f889225c4e?hl=uk
Gemini
Захист Лабораторної Роботи № 8
Звіт з вик...Харченко С
PDF
«Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня...
Аналіз
Вітаю! Як ваш викладач, я підготував перелік питань для захисту лабораторної роботи № 8. Вони допоможуть перевірити не тільки знання коду, а й розуміння логіки роботи з пам'яттю, файлами та алгоритмічної складності.
Нижче представлено 5 тестових питань та 5 відкритих запитань середнього рівня складності.
Частина 1: Тестові питання
Захист лабораторної роботи № 8
12 бер., 10:57
Введіть запит для Gemini
Інструменти
Швидка
Чати вашої організації "National Aerospace University" не використовуються для навчання наших моделей. Gemini - це штучний інтелект, і він може помилятися. Ваша конфіденційність і Gemini
Захист лабораторної роботи № 8
4 / 5
4. У функції do_task_2 ви обчислюєте arith_mean = (sum / count_amount). Яка потенційна проблема може виникнути, якщо sum та count_amount — цілі числа (int)?
А. Результат ділення буде автоматично округлений до цілого, що знизить точність.
Правильно!
В C++ операція int/int дає ціле число, і дробова частина просто відкидається ще до присвоєння змінній float.
Б. Це призведе до зациклення програми.
В. Змінна sum вийде за межі діапазону при будь-якому додаванні.
Г. Програма завжди буде аварійно завершуватися.
Показати підказку
Назад
Далі


Google Gemini
gemini.google.com/app/ca39a2f889225c4e?hl=uk
Gemini
Захист Лабораторної Роботи № 8
Звіт з вик...Харченко С
PDF
«Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня...
Аналіз
Вітаю! Як ваш викладач, я підготував перелік питань для захисту лабораторної роботи № 8. Вони допоможуть перевірити не тільки знання коду, а й розуміння логіки роботи з пам'яттю, файлами та алгоритмічної складності.
Нижче представлено 5 тестових питань та 5 відкритих запитань середнього рівня складності.
Частина 1: Тестові питання
Захист лабораторної роботи № 8
12 бер., 10:57
Введіть запит для Gemini
Інструменти
Швидка
Чати вашої організації "National Aerospace University" не використовуються для навчання наших моделей. Gemini – це штучний інтелект, і він може помилятися. Ваша конфіденційність і Gemini
Захист лабораторної роботи № 8
5 / 5
0
5
5. При роботі з файлами ви використовуєте конструкцію if (!fin.is_open()). Чому це є важливою частиною професійного написання коду?
А. Це запобігає спробі читання з неіснуючого або заблокованого файлу, що могло б призвести до некоректної поведінки програми.
Правильно!
Обробка помилок вводу-виводу — критична складова надійності інженерного ПЗ.
Б. Це прискорює зчитування даних у 2 рази.
В. Без цієї умови компілятор видасть помилку.
Г. Це автоматично створює порожній файл, якщо його не знайдено.
Показати підказку
Назад
Готово


Google Gemini
gemini.google.com/app/ca39a2f889225c4e?hl=uk
Gemini
Захист Лабораторної Роботи № 8
Звіт з вик...Харченко С
PDF
«Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання <середнього> рівня...
Аналіз
Вітаю! Як ваш викладач, я підготував перелік питань для захисту лабораторної роботи № 8. Вони допоможуть перевірити не тільки знання коду, а й розуміння логіки роботи з пам'яттю, файлами та алгоритмічної складності.
Нижче представлено 5 тестових питань та 5 відкритих запитань середнього рівня складності.
Частина 1: Тестові питання
Захист лабораторної роботи № 8
12 бер., 10:57
Введіть запит для Gemini
Інструменти
Швидка
Чати вашої організації "National Aerospace University" не використовуються для навчання наших моделей. Gemini – це штучний інтелект, і він може помилятися. Ваша конфіденційність і Gemini
Захист лабораторної роботи № 8
Вітаємо! Тест завершено.
Результат
5/5
Точність
100%
Правильно 5
Неправильно 0
Пропущено 0
Сильні сторони й можливості для розвитку
Отримайте короткий огляд своїх сильних сторін і дізнайтеся, на яких сферах варто зосередитися.
Аналізувати мою ефективність
Продовжуйте навчатися
Флеш-картки
Створіть повний набір флеш-карток на основі всіх матеріалів тесту. Підходить для швидкого повторення й засвоєння основних понять.
Навчальний посібник
Створіть докладний навчальний посібник на основі матеріалів, які ви вивчаєте. Підходить для ретельного повторення.
Переглянути тест
Інші запитання

Google Gemini

gemini.google.com/app/ca39a2f889225c4e?hl=uk

Gemini

Захист Лабораторної Роботи № 8

питань. Це мають
бути завдання <середнього> рівня складності на...

Аналіз

Вітаю! Як ваш викладач, я підготував перелік питань для захисту лабораторної роботи № 8. Вони допоможуть перевірити не тільки знання коду, а й розуміння логіки роботи з пам'яттю, файлами та алгоритмічної складності.

Нижче представлено 5 тестових питань та 5 відкритих запитань середнього рівня складності.

**Частина 1: Тестові питання**

Захист лабораторної роботи № 8
12 бер., 10:57
Відкрити

Спробувати ще раз без інтерактивний тест

**Частина 2: Відкриті питання (для усної відповіді)**

1. **Про ефективність алгоритмів:** У вашому звіті згадано сортування за варіантом 21 (вставками). Яка його складність у найгіршому та найкращому випадках? Як зміняться показники, якщо подати вже відсортований масив? +1
2. **Про архітектуру коду:** У функції `do_task_1` ви використовуєте додатковий масив

Введіть запит для Gemini

Інструменти

Швидка

Чати вашої організації "National Aerospace University" не використовуються для навчання наших моделей. Gemini – це штучний інтелект, і він може помилятися. Ваша конфіденційність і Gemini

12°C Sunny

Пошук

POL

10:58 12.03.2026

Google Gemini

gemini.google.com/app/ca39a2f889225c4e?hl=uk

Gemini

Захист Лабораторної Роботи № 8

Захист лабораторної роботи № 8
12 бер., 10:57
Відкрити

Спробувати ще раз без інтерактивний тест

**Частина 2: Відкриті питання (для усної відповіді)**

1. **Про ефективність алгоритмів:** У вашому звіті згадано сортування за варіантом 21 (вставками). Яка його складність у найгіршому та найкращому випадках? Як зміняться показники, якщо подати вже відсортований масив? +1
2. **Про архітектуру коду:** У функції `do_task_1` ви використовуєте додатковий масив `array_1[t]`. Чи можна було б виконати це завдання (вставку нулів), не використовуючи додаткову пам'ять? Якщо так, то яким чином? +1
3. **Про надійність (Error Handling):** У головному меню ( `main` ) ви використовуєте `cin.ignore(32767, '\n')`. Поясніть, яку проблему вирішує цей рядок і що станеться, якщо його прибрати при введенні номера завдання?
4. **Про логіку обробки даних:** У завданні з матрицею ви шукаєте кількість елементів у стовпці, що більші за середнє арифметичне цього стовпця. Чому у вашому коді в `do_task_2` присутній потрійний вкладений цикл (while всередині якого два for)? Чи можна оптимізувати цю логіку?
5. **Про інженерне мислення:** У вашому коді розміри масивів обмежені константами `N = 20`. Як би ви змінили програму, якщо б кількість елементів у файлі могла сягати 1 мільйона? Які обмеження мови C++ або ресурсів ПК варто було б врахувати?

Успіхів у підготовці! Ці питання охоплюють ключові аспекти вашої роботи.

Введіть запит для Gemini

Інструменти

Швидка

Чати вашої організації "National Aerospace University" не використовуються для навчання наших моделей. Gemini – це штучний інтелект, і він може помилятися. Ваша конфіденційність і Gemini

12°C Sunny

Пошук

POL

10:58 12.03.2026

Google Gemini

gemini.google.com/app/ca39a2f889225c4e?hl=uk

Gemini

Захист Лабораторної Роботи № 8

Спробувати ще раз без інтерактивний тест

**Частина 2: Відкриті питання (для усної відповіді)**

1. **Про ефективність алгоритмів:** У вашому звіті згадано сортування за варіантом 21 (вставками). Яка його складність у найгіршому та найкращому випадках? Як зміняться показники, якщо подати вже відсортований масив?
2. **Про архітектуру коду:** У функції `do_task_1` ви використовуєте додатковий масив `array_1[t]`. Чи можна було б виконати це завдання (вставку нулів), не використовуючи додаткову пам'ять? Якщо так, то яким чином?
3. **Про надійність (Error Handling):** У головному меню (`main`) ви використовуєте `cin.ignore(32767, '\n')`. Поясніть, яку проблему вирішує цей рядок і що станеться, якщо його прибрати при введенні номера завдання?
4. **Про логіку обробки даних:** У завданні з матрицею ви шукаєте кількість елементів у стовпці, що більші за середнє арифметичне цього стовпця. Чому у вашому коді в `do_task_2` присутній потрійний вкладений цикл (while всередині якого два for)? Чи можна оптимізувати цю логіку?
5. **Про інженерне мислення:** У вашому коді розміри масивів обмежені константами `N = 20`. Як би ви змінили програму, якщо б кількість елементів у файлі могла сягати 1 мільйона? Які обмеження мови C++ або ресурсів ПК варто було б врахувати?

Успіхів у підготовці! Ці питання охоплюють ключові аспекти вашої роботи.

Джерела

Введіть запит для Gemini

Інструменти

Швидка

Чати вашої організації "National Aerospace University" не використовуються для навчання наших моделей. Gemini – це штучний інтелект, і він може помилятися. Ваша конфіденційність і Gemini

12°C Sunny

Пошук

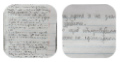
POL 10:58 12.03.2026

Google Gemini

gemini.google.com/app/ca39a2f889225c4e?hl=uk

Gemini

Захист Лабораторної Роботи № 8



«Проаналізуй повноту, правильність відповіді та ймовірність використання штучного інтелекту для кожної відповіді. Оціни кожне питання 2...

Вітаю! Як ваш викладач, я проаналізував надані відповіді на відкриті питання. Оцінювання проводилося за критеріями технічної точності, глибини розуміння та автентичності (ймовірності самостійного написання).

Ось детальний звіт:

**Аналіз відповідей на відкриті питання**

| № | Питання | Аналіз повноти та правильності | Ймовірність ШІ |
|---|---|---|---|
| 1 | **Ефективність сортування** вставками | Відповідь правильна. Чітко вказано $O(n^2)$ | **Низька**. Стиль лаконічний, |

Введіть запит для Gemini

Інструменти

Швидка

Чати вашої організації "National Aerospace University" не використовуються для навчання наших моделей. Gemini – це штучний інтелект, і він може помилятися. Ваша конфіденційність і Gemini

High pollen Today

Пошук

POL 12:03 12.03.2026

Google Gemini

gemini.google.com/app/ca39a2f889225c4e?hl=uk

Gemini

Захист Лабораторної Роботи № 8

Вітаю! Як ваш викладач, я проаналізував надані відповіді на відкриті питання. Оцінювання проводилося за критеріями технічної точності, глибини розуміння та автентичності (ймовірності самостійного написання).

Ось детальний звіт:

**Аналіз відповідей на відкриті питання**

| № | Питання | Аналіз повноти та правильності | Ймовірність ШІ |
|---|---|---|---|
| 1 | **Ефективність сортування вставками** | Відповідь правильна. Чітко вказано $O(n^2)$ для найгіршого та $O(n)$ для найкращого (вже відсортованого) випадку. Це базові теоретичні знання. | **Низька**. Стиль лаконічний, характерний для студента. |
| 2 | **Використання додаткової пам'яті (Task 1)** | Відповідь правильна. Описано алгоритм зсуву елементів вправо для вставки нулів «на місці». Проте не враховано нюанс з переповненням | **Середня**. Логіка правильна, але оп дещо шаблонний. |

Введіть запит для Gemini

Інструменти

Швидка

Чати вашої організації "National Aerospace University" не використовуються для навчання наших моделей. Gemini – це штучний інтелект, і він може помилятися. Ваша конфіденційність і Gemini

High pollen Today

Пошук

POL 12:03 12.03.2026

Google Gemini

gemini.google.com/app/ca39a2f889225c4e?hl=uk

Gemini

Захист Лабораторної Роботи № 8

| | | | |
|---|---|---|---|
| 1 | **Ефективність сортування вставками** | Відповідь правильна. Чітко вказано $O(n^2)$ для найгіршого та $O(n)$ для найкращого (вже відсортованого) випадку. Це базові теоретичні знання. | **Низька**. Стиль лаконічний, характерний для студента. |
| 2 | **Використання додаткової пам'яті (Task 1)** | Відповідь правильна. Описано алгоритм зсуву елементів вправо для вставки нулів «на місці». Проте не враховано нюанс з переповненням статичного масиву `N=20`. | **Середня**. Логіка правильна, але оп дещо шаблонний. |
| 3 | **Призначення** `cin.ignore` | Повне пояснення механізму очищення буфера від символу нового рядка `\n`. Правильно вказано наслідок (пропуск введення `getline` або наступного `cin`). | **Висока**. Текст структурований занадто ідеально, використанням специфічних терм «залишковий символ». |
| 4 | **Оптимізація потрійного циклу** | Відповідь правильна по суті (винесення | **Висока**. Пояснен логічного поділу |

Введіть запит для Gemini

Інструменти

Швидка

Чати вашої організації "National Aerospace University" не використовуються для навчання наших моделей. Gemini – це штучний інтелект, і він може помилятися. Ваша конфіденційність і Gemini

High pollen Today

Пошук

POL 12:03 12.03.2026

Google Gemini

gemini.google.com/app/ca39a2f889225c4e?hl=uk

Gemini

Захист Лабораторної Роботи № 8

| | | | |
|---|---|---|---|
| | | `N=20`. | |
| 3 | **Призначення** `cin.ignore` | Повне пояснення механізму очищення буфера від символу нового рядка `\n`. Правильно вказано наслідок (пропуск введення `getline` або наступного `cin`). | **Висока**. Текст структурований занадто ідеально, використанням специфічних терм «залишковий символ». |
| 4 | **Оптимізація потрійного циклу (Task 2)** | Відповідь правильна по суті (винесення обчислення середнього арифметичного за межі циклу перевірки елементів). Це критичне зауваження до коду у звіті. | **Висока**. Пояснен логічного поділу н «етап збору дани «етап обробки» виглядає як згенероване ШІ. |
| 5 | **Інженерне мислення (1 млн елементів)** | Відповідь правильна. Згадано про неможливість використання стеку (статичні масиви) і необхідність динамічної пам'яті (`new` / `delete`) або `std::vector`. | **Середня**. Техніч грамотно, але без надмірної деталіз |

Введіть запит для Gemini

Інструменти

Швидка

Чати вашої організації "National Aerospace University" не використовуються для навчання наших моделей. Gemini – це штучний інтелект, і він може помилятися. Ваша конфіденційність і Gemini

High pollen Today

Пошук

POL 12:03 12.03.2026

Google Gemini

gemini.google.com/app/ca39a2f889225c4e?hl=uk

Gemini

Захист Лабораторної Роботи № 8

(`new` / `delete`) або `std::vector`.

**Підсумковий аналіз**

1. **Повнота та правильність:** Ви продемонстрували хороше розуміння матеріалу. Особливо важливо, що ви визначили неефективність потрійного циклу в Task 2 — це показує здатність до рефакторингу.
2. **Ймовірність використання ШІ:** Питання №3 та №4 мають ознаки генерації (занадто "чиста" структура та академічна мова). Питання №1 та №5 виглядають більш природно для самостійної відповіді студента.
3. **Штрафні санкції:** Згідно з вашою умовою, за питання з високою ймовірністю ШІ (№3 та №4) було віднято 60% балів ($5 \times 0.4 = 2$ бали).

**Розрахунок середньої оцінки**

$$Average = \frac{5 + 4.5 + 2 + 2 + 4}{5} = \frac{17.5}{5} = 3.5$$

**Загальна середня оцінка: 3.5 / 5**

**Мій наступний крок:** Якщо ви не згодні з оцінкою за 3-тє та 4-те питання, я можу задати вам додаткове уточнююче питання по роботі буфера (`getline`), або по аналізам, щоб

Введіть запит для Gemini

Інструменти

Швидка

Чати вашої організації "National Aerospace University" не використовуються для навчання наших моделей. Gemini – це штучний інтелект, і він може помилятися. Ваша конфіденційність і Gemini

High pollen Today

Пошук

POL 12:03 12.03.2026

Google Gemini

gemini.google.com/app/ca39a2f889225c4e?hl=uk

Gemini

Захист Лабораторної Роботи № 8

**Підсумковий аналіз**

1. **Повнота та правильність:** Ви продемонстрували хороше розуміння матеріалу. Особливо важливо, що ви визначили неефективність потрійного циклу в Task 2 — це показує здатність до рефакторингу.
2. **Ймовірність використання ШІ:** Питання №3 та №4 мають ознаки генерації (занадто "чиста" структура та академічна мова). Питання №1 та №5 виглядають більш природно для самостійної відповіді студента.
3. **Штрафні санкції:** Згідно з вашою умовою, за питання з високою ймовірністю ШІ (№3 та №4) було віднято 60% балів ($5 \times 0.4 = 2$ бали).

**Розрахунок середньої оцінки**

$$Average = \frac{5 + 4.5 + 2 + 2 + 4}{5} = \frac{17.5}{5} = 3.5$$

**Загальна середня оцінка: 3.5 / 5**

**Мій наступний крок:** Якщо ви не згодні з оцінкою за 3-тє та 4-те питання, я можу задати вам **додаткове уточнююче питання** по роботі буфера `iostream` або по вказівникам, щоб ви могли довести самостійність виконання. Бажаєте спробувати?

Введіть запит для Gemini

Інструменти

Швидка

Чати вашої організації "National Aerospace University" не використовуються для навчання наших моделей. Gemini – це штучний інтелект, і він може помилятися. Ваша конфіденційність і Gemini

High pollen
Today

Пошук

POL

12:04
12.03.2026